

Reviewer Pack — Minimal Rank of p-Adic Derivatives and Average-Case Circuit ...

Entry eaae7183dcc9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 11:53:28 UTC

Minimal Rank of p-Adic Derivatives and Average-Case Circuit Size

Entry ID: eaae7183dcc9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 11:53:28 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Average-case Circuit Complexity

Statement:

{‘st1’: ‘For every language L in NP, the average-case circuit size for L is polynomially bounded by the minimal rank of its p-adic derivative.’, ‘st2’: ‘Specifically, $E[\text{CircuitSize}(L)] = O(\text{Rank}(\Delta L))$ where ΔL is the p-adic derivative of the characteristic function of L .’, ‘st3’: ‘The above holds for all instances with size $n \leq 40$.’}

Rationale (proposer’s reasoning):

{‘st1’: ‘p-adic analysis provides a framework to study the arithmetic properties of functions, which might reveal subtle structures in the computation process that affect average-case complexity.’, ‘st2’: ‘Since p-adic derivatives can capture information about local behavior, they could be related to the complexity of average-case algorithms, especially for hard problems.’, ‘st3’: ‘This connection could lead to new insights into the nature of NP-hardness and average-case complexity.’}

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c2eb53fc67bc5d46

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average-case circuit size for instances in NP with size $n \leq 40$ is polynomially bounded by the minimal rank of their p-adic derivatives, as evidenced by at least 24 out of 30 seeds showing a correlation coefficient above 0.9 between the rank of the p-adic derivative and the circuit size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=15.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def p_adic_derivative(f):
        n = len(f)
        if n <= 1:
            return []
        derivative = [0] * (n - 1)
        for i in range(n - 1):
            derivative[i] = f[i + 1] - f[i]
        return derivative

    def circuit_complexity(f):
        n = len(f)
        if n == 1:
            return 1
        if all(x == 0 for x in f) or all(x == 1 for x in f):
            return 1
        min_circuit_size = float('inf')
        for i in range(1, n):
            left = circuit_complexity(f[:i])
            right = circuit_complexity(f[i:])
            min_circuit_size = min(min_circuit_size, left + right + 1)
        return min_circuit_size

    instances_tested = 0
    total_rank = 0
```

```

total_circuit_size = 0

for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    rank = len(p_adic_derivative(f))
    circuit_size = circuit_complexity(f)

    if rank == 0 or circuit_size == 0:
        continue

    total_rank += rank
    total_circuit_size += circuit_size
    instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

mean_rank = total_rank / instances_tested
mean_circuit_size = total_circuit_size / instances_tested

correlation_coefficient = (instances_tested * mean_rank * mean_circuit_size -
                           sum(rank * circuit_size for rank, circuit_size in zip(p_adic_derivative(f),
                                                                                  circuit_complexity(f)))) /
    math.sqrt((instances_tested * sum(rank**2 for rank in p_adic_derivative(f)) - sum(rank**2 for rank in range(1, 41))) *
              (instances_tested * sum(circuit_size**2 for circuit_size in circuit_complexity(f)) - sum(circuit_size**2 for circuit_size in range(1, 41))))

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "conjecture_holds": correlation_coefficient > 0.9,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["metric_value"] is not None) / (len(results) - 1))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) <= 0.2:
        print(f"RESULT: NOT SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        print(f"RESULT: UNKNOWN mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 93, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 54, in run_trial
    circuit_size = circuit_complexity(f)
                  ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 42, in circuit_complexity
    right = circuit_complexity(f[i:])
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 42, in circuit_complexity
    right = circuit_complexity(f[i:])
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 42, in circuit_complexity
    right = circuit_complexity(f[i:])
            ^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 994 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f2edede.py", line 37, in circuit_complexity
    if all(x == 0 for x in f) or all(x == 1 for x in f):
       ^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased recursion limits and ensure that it completes without crashing. If the test passes, re-evaluate the conjecture based on the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14679
2	preregistration	ollama_remote	glm4:latest	0	0	9719
3	novelty	ollama_remote	glm4:latest	0	0	8388
4	novelty	ollama_remote	glm4:latest	0	0	8385
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14039
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11479
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12990
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12874
9	judge	ollama_remote	glm4:latest	0	0	33966

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126519 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eaee7183dcc9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eaee7183dcc9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eaee7183dcc9.tar.gz` (if generated)